

Assignment 3 Report

LLM Offloading and Memory Hierarchy Observation
NYCU [CSIC30155] Memory and Storage System

Student ID: your_student_id

Due: June 1, 2026

Hardware Information

Component	Details
GPU	NVIDIA GeForce RTX 5080 (16 GB VRAM, 360 W TDP)
CPU	Intel Core Ultra 9 285K (24 cores, no hyper-threading)
RAM	125 GB
Disk	NVMe SSD (nvme1n1) mounted at /, ~537 GB free on 915 GB partition
Path	Local (not Colab)
Model	facebook/opt-1.3b (weights at ~/opt_weights/opt-1.3b-np/)
FlexGen	FlexLLMGen v0.1.7, PyTorch 2.6.0+cu124, CUDA 12.4 / Driver 595.71

1 Q1 (30%): Progressive Offload Sweep

Five weight distributions are swept with fixed flags:

--gpu-batch-size 4 --prompt-len 128 --gen-len 16.

Q1.1 Throughput and Peak GPU Memory

Table 1: Q1 results: total throughput and peak GPU memory for the five datapoints.

Set.	Weight Distribution	-percent						Throughput (tok/s)	Peak GPU Mem (GB)
		(W_g K_c	W_c A_g	K_g A_c)					
(1)	100% GPU	100	0	100	0	100	0	277.07	2.829
(2)	50% GPU + 50% CPU	50	50	100	0	100	0	95.74	1.705
(3)	50% GPU + 50% Disk	50	0	100	0	100	0	23.06	1.737
(4)	50% CPU + 50% Disk	0	50	100	0	100	0	20.75	0.610
(5)	100% Disk	0	0	100	0	100	0	14.67	0.610

Q1.2 Boundary Ratio Analysis

Table 2: Throughput ratio and dominant bottleneck at each tier boundary.

Boundary	Ratio	Value	Dominant Bottleneck
(1) → (2)	277.07/95.74	$\approx 2.89\times$	PCIe
(2) → (3)	95.74/23.06	$\approx 4.15\times$	Disk
(3) → (4)	23.06/20.75	$\approx 1.11\times$	PCIe
(4) → (5)	20.75/14.67	$\approx 1.41\times$	Disk

(a) Within-class comparison. PCIe boundaries: The front-pair PCIe ratio ((1) → (2) = $2.89\times$) is much larger than the back-pair PCIe ratio ((3) → (4) = $1.11\times$). At (1) → (2), the baseline is the fastest possible configuration (all weights live in GPU HBM). Moving half the weights to CPU forces those weights to traverse the PCIe bus on every token step, introducing a severe bottleneck for what was previously zero-latency access. At (3) → (4), the system is already bottlenecked by the 50% of weights on disk. Moving the GPU half to CPU adds PCIe overhead for that portion, but the disk-reading half dominates; the incremental penalty from the newly-PCIe-bound half is therefore much smaller.

Disk boundaries: The front-pair Disk ratio ((2) → (3) = $4.15\times$) is also larger than the back-pair Disk ratio ((4) → (5) = $1.41\times$). At (2) → (3), the CPU half is switched to disk, replacing PCIe-bounded reads with much slower disk reads. NVMe sequential read bandwidth ($\sim 3\text{--}7$ GB/s) is roughly one order of magnitude below PCIe Gen 5 x16 bandwidth (~ 64 GB/s), so this switch causes a large degradation. At (4) → (5), the

system is already 50% disk-bound; switching the remaining CPU half to disk adds more disk I/O, but the system can overlap some reads with the already-running disk transfers, reducing the incremental slowdown.

(b) Cross-class within-pair comparison. **Front pair** ((1) $\rightarrow$ (2) PCIe vs. (2) $\rightarrow$ (3) Disk): Within the front pair, the Disk ratio ($4.15\times$) exceeds the PCIe ratio ($2.89\times$). Because NVMe disk bandwidth is $\sim 1\text{--}2$ orders of magnitude below both GPU HBM and PCIe bandwidth, replacing CPU-resident (PCIe-bounded) access with disk access is more damaging than replacing GPU-resident access with CPU-resident (PCIe-bounded) access.

Back pair ((3) $\rightarrow$ (4) PCIe vs. (4) $\rightarrow$ (5) Disk): Again the Disk ratio ($1.41\times$) is larger than the PCIe ratio ($1.11\times$) for the same bandwidth-ordering reason: disk $\ll$ PCIe. The absolute values are smaller in the back pair because the overall system is already deeply I/O-bound.

2 Q2 (20%): Batch Size and I/O Amortization

100% disk offload (`--percent 0 0 100 0 100 0`) with varying `--gpu-batch-size`.

Q2.1 Wall Time, Throughput, and Peak GPU Memory

Table 3: Q2 results across three batch sizes (100% disk offload).

Batch	Wall Time (s)	Throughput (tok/s)	Peak GPU Mem (GB)
$b = 1$	4.430	3.611	0.493
$b = 4$	4.362	14.673	0.610
$b = 16$	4.393	58.277	1.072

Q2.2 Throughput Ratios and Wall Time Analysis

$$\frac{b_4}{b_1} = \frac{14.673}{3.611} \approx 4.06\times, \quad \frac{b_{16}}{b_4} = \frac{58.277}{14.673} \approx 3.97\times$$

Both ratios are very close to the theoretical $4\times$. This is explained by the wall time: all three runs complete in nearly the same total time (≈ 4.4 s). In disk-offload mode the bottleneck is weight loading—for every decode step the entire model (~ 2.4 GB) must be streamed from disk, independent of how many prompts are processed simultaneously. Because the number of disk reads per step is fixed, wall time is nearly constant across batch sizes. Throughput (tokens per second) is therefore $\propto$ batch size/wall time $\approx$ batch size/ C , yielding an almost perfect linear scaling with batch size, which is why both ratios are $\approx 4\times$.

Q2.3 Can Batch Size Be Scaled Indefinitely?

No. Peak GPU memory grows with batch size (0.493 GB $\rightarrow$ 0.610 GB $\rightarrow$ 1.072 GB for $b = 1, 4, 16$). The KV cache grows linearly with batch size (one cache entry per prompt per layer), and all KV cache is held on the GPU in these runs (100 0 for KV cache). Once the combined memory of the KV cache plus the single-layer weight shard resident on GPU exceeds the 16 GB VRAM, FlexGen will run out of memory. For the same reason, the increase per $4\times$ batch step is sublinear at small batch (0.493 $\rightarrow$ 0.610 GB), because a fixed base occupancy (activation buffers, etc.) is amortized across more requests; however, at very large batches the KV cache term dominates and further scaling would eventually OOM.

3 Q3 (20%): Weight Compression and Tier Interaction

`--compress-weight` quantizes weights from FP16 to 4-bit ($4\times$ compression); dequantization occurs at load time before feeding the GPU.

Q3.1 Throughput and Peak GPU Memory

Table 4: Q3 results: throughput and peak GPU memory with/without `--compress-weight`.

Scenario	Compress?	Throughput (tok/s)	Peak GPU Mem (GB)
(α) 100% GPU	No	277.07	2.829
(α) 100% GPU	Yes	122.17	1.125
(β) 100% Disk	No	14.67	0.610
(β) 100% Disk	Yes	19.56	0.491

Q3.2 Analysis

(a) Costs reduced, costs added, and which dominates. **Compression reduces:** the number of bytes that must be transferred from the storage tier to GPU. With 4-bit quantization, weights occupy $4\times$ less space, so $4\times$ less data must traverse PCIe (CPU $\rightarrow$ GPU) or be read from disk.

Compression adds: a dequantization kernel that runs on the GPU before each compute pass, consuming GPU cycles.

Scenario (α) — 100% GPU: All weights already reside in GPU VRAM; there is no inter-tier data transfer during inference. The “reduced transfer” benefit is irrelevant here. The dequantization overhead is *pure extra work* layered on top of the matmul compute. Result: throughput *drops* from $277 \rightarrow 122$ tok/s (roughly $2.3\times$ slower). Compute overhead dominates.

Scenario (β) — 100% Disk: The bottleneck without compression is disk I/O: each decode step must read ~ 2.4 GB from NVMe. With 4-bit compression the same information is stored in ~ 0.6 GB, so disk reads are $4\times$ smaller. The dequantization overhead on GPU is small relative to the disk read time saved. Result: throughput *increases* from $14.67 \rightarrow 19.56$ tok/s ($\approx 1.33\times$ faster). I/O savings dominate.

(b) Why is the magnitude not close to the $4\times$ compression ratio? In scenario (α), the slowdown is only $2.3\times$ (not $4\times$) because dequantization is a lightweight element-wise kernel—much faster than the weight-matrix multiplications that follow. The compute bottleneck was already the matmul; adding dequantization increases GPU occupancy but not by $4\times$.

In scenario (β), the speedup is only $1.33\times$ (not $4\times$) because disk I/O is not the *only* bottleneck. As seen in the Q4 Part 2 breakdown, the per-layer compute time and the per-layer load time are of similar magnitude for the disk case (load_weight = 5.538 ms,

compute = 5.079 ms). Compressing weights reduces the load time by $\approx 4\times$ (to ~ 1.4 ms), but now GPU compute becomes the new bottleneck (~ 5 ms per layer). The effective time per layer therefore converges towards the compute floor, yielding a speedup much smaller than $4\times$.

4 Q4 (30%): I/O Behavior and Bottleneck Analysis under Disk Offload

Q4.1 (15%): Part 1 — Disk I/O Pattern

Disk holding weight files: `nvme1n1` (the root NVMe SSD, mounted at `/`).

Table 5: Peak `iostat -x 1` values for `nvme1n1` during the 100% disk offload run.

iostat column	Peak value	Unit
r/s	59.49	req/s
rkB/s	3,446	KB/s
rareq-sz	57.93	KB/req
%util	40.5	%

Classification: Pattern A — Sequential, large blocks.

The peak average request size of 57.93 KB/req is well above the 32 KB threshold. This indicates that the kernel’s readahead mechanism detected a sequential access pattern and batched adjacent pages into large coalesced requests before they reached the block layer. The comparatively low request rate (59 req/s) and moderate utilization (40.5%) are also consistent with a sequential workload: data is read in large, contiguous chunks rather than many independent random requests.

FlexGen reads the numpy weight shards sequentially (each shard is a contiguous file stored by `np.save`), which naturally triggers kernel readahead and produces the large-block signature observed.

Q4.2 (15%): Part 2 — Stage Timings (`--debug-mode` breakdown)

Table 6: Per-stage timing from `--debug-mode` breakdown (seconds; 20 batches).

Scenario	load_wt (s)	compute (s)	load/compute
(1) Baseline (100% GPU)	0.000011	0.005151	0.0021
(5) 100% Disk offload	0.005538	0.005079	1.090

(a) Comparing `compute_layer_decoding` across scenarios. The per-batch compute time is nearly identical in both scenarios (0.005151 s vs. 0.005079 s, a difference of $< 2\%$). This is expected: offloading changes *where* weights are stored but not *how many multiply-accumulate operations* the GPU performs per token. Every decode step still multiplies the same activations against the same weight tensors; the GPU arithmetic is unchanged. Offloading therefore has no effect on GPU compute throughput; the only thing it changes is how long the GPU must wait for weights to arrive.

(b) Comparing load_weight across scenarios. The load_weight time increases from 0.000011 s (baseline) to 0.005538 s (disk offload)—a factor of

$$\frac{0.005538}{0.000011} \approx 503\times,$$

roughly **two to three orders of magnitude**.

In the baseline (100% GPU), the weights are already resident in GPU memory; “loading” a layer amounts to advancing a pointer, essentially free. In the disk offload case, each layer’s weights (~ 2.44 GB / 50 layers ≈ 49 MB per layer) must be read from the NVMe drive, moved through the PCIe bus, and placed into GPU memory before the compute kernel can run. This is purely a *storage/interconnect* operation—it is bottlenecked by disk read bandwidth and PCIe transfer bandwidth, not by GPU arithmetic. The $\sim 500\times$ increase in load time directly reflects the bandwidth gap between GPU HBM (used in the baseline) and the disk-PCIe path used under offload. The load/compute ratio jumping from 0.002 (compute-bound) to 1.09 (I/O-bound, near parity) confirms that disk offload turns a compute-bound workload into an I/O-bound one.